Lógica de Programação | Aula 12

Funções e Escopo em Python

João Quentino

13 de Março de 2026

Jovem Tech | Turma 04A - Manhã

Recapitulação

O que já aprendemos:

Estruturas de Controle:

- if/elif/else
- for e while
- break e continue

Estruturas de Dados:

- Listas
- Strings
- Tipos básicos

Operações:

- Aritméticas
- Lógicas
- Relacionais
- Manipulação de listas

Hoje:

- Organizar código
- Reutilizar lógica

Agenda de Hoje

Introdução às Funções

Criando Funções

Parâmetros e Argumentos

Retorno de Valores

Escopo de Variáveis

Docstrings e Documentação

Exercícios Práticos

Projeto Integrador

Boas Práticas

Recapitulação

Introdução às Funções

Por Que Precisamos de Funções?

Problema: Código repetitivo e desorganizado

Imagine calcular a área de vários retângulos:

- Retângulo 1: `area1 = 5 * 3`
- Retângulo 2: `area2 = 8 * 4`
- Retângulo 3: `area3 = 10 * 6`
- ...e assim por diante

Problemas:

- Código repetitivo
- Difícil de manter
- Se mudar a fórmula, precisa alterar em todos os lugares
- Impossível reutilizar em outros projetos

Solução: Funções!

Com funções, escrevemos uma vez e usamos várias:

```python
def calcular_area(largura, altura):
    area = largura * altura
    return area

# Usar a funcao
area1 = calcular_area(5, 3)
area2 = calcular_area(8, 4)
area3 = calcular_area(10, 6)
```

Vantagens:

- Código limpo e organizado
- Fácil de manter e modificar
- Reutilizável
- Testável

O que são Funções?

Definição:

Função é um bloco de código nomeado que realiza uma tarefa específica e pode ser reutilizado.

Analogia do mundo real:

- **Liquidificador**: recebe ingredientes, processa, devolve suco
- **Calculadora**: recebe números, opera, retorna resultado
- **Forno**: recebe massa, assa, retorna pão

Funções em programação:

- **Entrada**: parâmetros (ingredientes)
- **Processamento**: corpo da função (receita)
- **Saída**: valor de retorno (resultado)

Funções que Você Já Conhece!

Python tem muitas funções built-in:

Funções básicas:

- `print()` - exibe na tela
- `input()` - lê entrada
- `len()` - tamanho
- `type()` - tipo de dado
- `int()`, `float()`, `str()` - conversão

Funções matemáticas:

- `abs()` - valor absoluto
- `max()` - maior valor
- `min()` - menor valor
- `sum()` - soma
- `round()` - arredondar

Agora você aprenderá a criar suas próprias funções!

Criando Funções

Anatomia de uma Função

Estrutura básica:

```python
def nome_da_funcao(parametro1, parametro2):
    """Docstring: descricao da funcao (opcional)"""
    # Corpo da funcao
    resultado = parametro1 + parametro2
    return resultado
```

Componentes:

- `def` - palavra-chave que inicia definição
- `nome_da_funcao` - nome descritivo
- `(parametros)` - entrada da função
- `:` - dois pontos obrigatórios
- **Indentação** - corpo da função
- `return` - devolve resultado (opcional)

Primeira Função: Saudação

Função simples sem parâmetros:

```python
def saudacao():
    print("Ola, bem-vindo!")
    print("Como vai?")

# Chamar a funcao
saudacao()
```

Saída:

```
Ola, bem-vindo!
Como vai?
```

Importante:

- Definir não executa a função

Função com Parâmetro

Personalizar comportamento com parâmetros:

```python
def saudacao_personalizada(nome):
    print(f"Ola, {nome}!")
    print("Como vai?")

# Chamar com diferentes argumentos
saudacao_personalizada("Ana")
saudacao_personalizada("Bruno")
```

Saída:

```
Ola, Ana!
Como vai?
Ola, Bruno!
Como vai?
```

Parâmetros e Argumentos

Parâmetros vs Argumentos

Diferença importante:

Parâmetros

- Variáveis na **definição**
- "Placeholders"
- Como está declarado

```
def soma(a, b):
```

Argumentos

- Valores na **chamada**
- Valores reais
- Dados passados

```
soma(5, 3)
```

Resumo: Parâmetros são as variáveis, argumentos são os valores.

Múltiplos Parâmetros

Funções podem receber vários parâmetros:

```python
def calcular_imc(peso, altura):
    imc = peso / (altura ** 2)
    print(f"Peso: {peso} kg")
    print(f"Altura: {altura} m")
    print(f"IMC: {imc:.2f}")

# Chamadas
calcular_imc(70, 1.75)
calcular_imc(85, 1.80)
```

Importante:

- Argumentos são passados na **ordem**
- Número de argumentos deve coincidir com número de parâmetros

Argumentos Posicionais vs Nomeados

Posicionais:

```python
def apresentar(nome, idade):
    print(f"{nome}, {idade} anos")

# Ordem importa!
apresentar("Ana", 25)
# Ana, 25 anos

apresentar(25, "Ana")
# 25, Ana anos (errado!)
```

Nomeados:

```python
def apresentar(nome, idade):
    print(f"{nome}, {idade} anos")

# Ordem nao importa
apresentar(idade=25, nome="Ana")
# Ana, 25 anos

apresentar(nome="Bruno", idade=30)
# Bruno, 30 anos
```

Dica: Use nomeados para clareza, especialmente com muitos parâmetros

Parâmetros com Valores Padrão

Definir valores padrão para parâmetros opcionais:

```python
def saudacao(nome, saudacao="Ola", pontuacao="!"):
    mensagem = f"{saudacao}, {nome}{pontuacao}"
    print(mensagem)

# Usando todos os padroes
saudacao("Ana")  # Ola, Ana!

# Mudando saudacao
saudacao("Bruno", "Bem-vindo")  # Bem-vindo, Bruno!

# Mudando pontuacao
saudacao("Carlos", pontuacao=".")  # Ola, Carlos.

# Mudando ambos
saudacao("Diana", "Oi", "!!!")  # Oi, Diana!!!
```

Retorno de Valores

O Comando return

return devolve um valor da função:

```python
def somar(a, b):
    resultado = a + b
    return resultado  # Devolve o valor

# Capturar o retorno
soma = somar(5, 3)
print(soma)  # 8

# Usar diretamente
print(somar(10, 20))  # 30

# Em expressoes
dobro_da_soma = somar(4, 6) * 2
print(dobro_da_soma)  # 20
```

Importante: `return` encerra a execução da função!

Função sem return

Funções sem return retornam None:

```python
def exibir_mensagem(texto):
    print(f"Mensagem: {texto}")
    # Sem return!

# Chamar
resultado = exibir_mensagem("Teste")
print(f"Retorno: {resultado}")  # Retorno: None
```

Quando usar return:

- Calcular e devolver resultado
- Processar dados e retornar transformação
- Validações (retornar True/False)

Quando NÃO usar:

Múltiplos Retornos

Retornar múltiplos valores (tupla):

```python
def dividir_com_resto(dividendo, divisor):
    quociente = dividendo // divisor
    resto = dividendo % divisor
    return quociente, resto  # Retorna tupla

# Desempacotar
q, r = dividir_com_resto(17, 5)
print(f"17 / 5 = {q} com resto {r}")  # 3 com resto 2
```

Outro exemplo:

```python
def estatisticas(numeros):
    return min(numeros), max(numeros), sum(numeros)/len(numeros)

minimo, maximo, media = estatisticas([1, 2, 3, 4, 5])
```

return Antecipado

return encerra a função imediatamente:

```python
def eh_par(numero):
    if numero % 2 == 0:
        return True
    return False
    # Se chegou aqui, nao e par

# Validacao com retorno antecipado
def dividir(a, b):
    if b == 0:
        print("Erro: divisao por zero!")
        return None  # Sai antes

    return a / b

resultado = dividir(10, 0)  # Erro: divisao por zero!
print(resultado)  # None
```

Escopo de Variáveis

O que é Escopo?

Escopo = onde uma variável existe e pode ser acessada

Tipos de escopo:

- **Local**: dentro da função
  - Existe apenas durante execução da função
  - Não acessível fora da função
- **Global**: fora de funções
  - Existe durante todo o programa
  - Acessível de qualquer lugar

Regra de ouro: O que acontece na função, fica na função! (a menos que retorne)

Escopo Local

Variáveis criadas dentro da função são locais:

```
def calcular():
    x = 10  # Variavel LOCAL
    y = 20  # Variavel LOCAL
    resultado = x + y
    return resultado

soma = calcular()
print(soma)  # 30

print(x)  # ERRO! x nao existe aqui
```

Por quê?

- `x` e `y` existem apenas dentro de `calcular()`
- Quando a função termina, variáveis locais são destruídas
- Isso evita conflitos de nomes

Escopo Global

Variáveis fora de funções são globais:

```python
contador = 0  # Variavel GLOBAL

def incrementar():
    print(f"Contador dentro: {contador}")  # Pode LER

incrementar()  # Contador dentro: 0
print(f"Contador fora: {contador}")  # Contador fora: 0
```

Importante:

- Funções podem **ler** variáveis globais
- Mas não podem **modificar** sem `global`
- (Veremos isso a seguir)

Modificando Variáveis Globais

Tentativa de modificação cria variável local:

```
contador = 0

def incrementar_errado():
    contador = contador + 1  # ERRO!
    # Tenta criar local antes de ler global

# Para modificar global, use palavra-chave global
def incrementar_certo():
    global contador
    contador = contador + 1

incrementar_certo()
print(contador)  # 1
```

Boa prática: Evite modificar globais! Use parâmetros e return.

Melhor Abordagem: Sem Globais

Ruim (com global):

```python
contador = 0

def incrementar():
    global contador
    contador += 1

incrementar()
incrementar()
print(contador)  # 2
```

Código difícil de entender

Bom (sem global):

```python
def incrementar(valor):
    return valor + 1

contador = 0
contador = incrementar(contador)
contador = incrementar(contador)
print(contador)  # 2
```

Código claro e previsível

Princípio: Funções devem receber dados por parâmetros e devolver por return

Docstrings e Documentação

Documentando Funções

Docstrings explicam o que a função faz:

```python
def calcular_area_retangulo(largura, altura):
    """
    Calcula a area de um retangulo.

    Parametros:
        largura (float): Largura do retangulo
        altura (float): Altura do retangulo

    Retorna:
        float: Area do retangulo
    """
    return largura * altura

# Acessar documentacao
help(calcular_area_retangulo)
```

Benefícios: Código autodocumentado, fácil de entender e usar

Exercícios Práticos

Exercício 1: Calculadora Simples

Desafio (10 min):

Crie 4 funções para operações básicas:

```python
def somar(a, b):
    # Complete...

def subtrair(a, b):
    # Complete...

def multiplicar(a, b):
    # Complete...

def dividir(a, b):
    # Complete (trate divisao por zero!)
# Teste
print(somar(5, 3))         # 8
print(dividir(10, 2))      # 5.0
print(dividir(10, 0))      # Erro!
```

Exercício 2: Validador de Senha

Desafio (15 min):

Crie função que valida senha:

```python
def validar_senha(senha):
    """
    Valida se senha atende criterios:
    - Minimo 8 caracteres
    - Pelo menos 1 numero
    - Pelo menos 1 letra maiuscula

    Retorna True se valida, False caso contrario
    """
    # Complete...

# Testes
print(validar_senha("abc123"))       # False (curta)
print(validar_senha("abcdefgh"))     # False (sem numero)
print(validar_senha("Abc12345"))     # True
```

Exercício 3: Conversor de Temperatura

Desafio (10 min):

Crie funções de conversão:

```python
def celsius_para_fahrenheit(celsius):
    # Formula: F = (C * 9/5) + 32
    # Complete...

def fahrenheit_para_celsius(fahrenheit):
    # Formula: C = (F - 32) * 5/9
    # Complete...

# Testes
print(celsius_para_fahrenheit(0))     # 32.0
print(celsius_para_fahrenheit(100))   # 212.0
print(fahrenheit_para_celsius(32))    # 0.0
```

Exercício 4: Estatísticas de Lista

Desafio (15 min):

Crie função que retorna estatísticas:

```python
def estatisticas(numeros):
    """
    Retorna minimo, maximo, media de uma lista.

    Retorna:
        tupla: (minimo, maximo, media)
    """
    # Complete...

# Teste
numeros = [10, 20, 30, 40, 50]
min_val, max_val, media = estatisticas(numeros)
print(f"Min: {min_val}, Max: {max_val}, Media: {media}")
# Min: 10, Max: 50, Media: 30.0
```

Exercício 5: Número Primo

Desafio (15 min):

Crie função que verifica se número é primo:

```python
def eh_primo(numero):
    """
    Verifica se numero e primo.

    Numero primo: divisivel apenas por 1 e ele mesmo
    """
    if numero < 2:
        return False

    # Complete o resto...

# Testes
print(eh_primo(2))     # True
print(eh_primo(17))    # True
print(eh_primo(20))    # False
```

Exercício 6: Fatorial Recursivo (Desafio)

Desafio avançado (20 min):

Crie função recursiva para fatorial:

```python
def fatorial(n):
    """
    Calcula fatorial de n usando recursao.

    Recursao: funcao que chama a si mesma
    Caso base: fatorial(0) = 1
    Caso recursivo: fatorial(n) = n * fatorial(n-1)
    """
    # Complete...

# Testes
print(fatorial(5))   # 120
print(fatorial(0))   # 1
print(fatorial(10))  # 3628800
```

Projeto Integrador

Projeto: Sistema de Notas Modular

Refatore o sistema de notas usando funções:

Funções necessárias:

1. `adicionar_nota(notas, nota)` - adiciona nota validada
2. `calcular_media(notas)` - retorna média
3. `contar_aprovados(notas, nota_minima)` - conta aprovados
4. `exibir_estatisticas(notas)` - mostra min, max, média
5. `menu_principal()` - exibe menu e retorna opção

Requisitos:

- Cada função deve fazer UMA coisa
- Use docstrings
- Validações apropriadas
- Menu interativo usando as funções

Projeto: Estrutura Inicial

```python
def adicionar_nota(notas, nota):
    """Adiciona nota se valida (0-10)."""
    if 0 <= nota <= 10:
        notas.append(nota)
        return True
    return False

def calcular_media(notas):
    """Calcula media das notas."""
    if len(notas) == 0:
        return 0
    return sum(notas) / len(notas)

def menu_principal():
    """Exibe menu e retorna opcao escolhida."""
    print("\n=== SISTEMA DE NOTAS ===")
    print("1 - Adicionar nota")
    print("2 - Ver estatisticas")
    print("0 - Sair")
    return input("Escolha: ")
```

Boas Práticas

Boas Práticas com Funções

Diretrizes essenciais:

1. **Faça uma coisa bem feita**
   - Cada função deve ter responsabilidade única
   - Se faz muitas coisas, divida em funções menores
2. **Nomes descritivos**
   - Use verbos: `calcular`, `validar`, `processar`
   - Seja específico: `calcular_imc` melhor que `calc`
3. **Evite efeitos colaterais**
   - Não modifique variáveis globais
   - Use parâmetros e return
4. **Documente**
   - Use docstrings
   - Explique parâmetros e retorno

Tamanho Ideal de Funções

Quanto código uma função deve ter?

Regra geral: Se cabe na tela sem scroll, está bom!

- **Ideal**: 5-15 linhas
- **Aceitável**: até 30 linhas
- **Problemático**: mais de 50 linhas

Se a função está muito grande:

- Identifique blocos lógicos
- Extraia para funções separadas
- Melhora legibilidade e testabilidade

Exemplo: Refatoração

Antes (tudo junto):

```python
def processar_aluno():
    nome = input("Nome: ")
    n1 = float(input("Nota 1: "))
    n2 = float(input("Nota 2: "))
    media = (n1 + n2) / 2

    if media >= 7:
        status = "Aprovado"
    else:
        status = "Reprovado"

    print(f"{nome}: {media}")
    print(f"Status: {status}")
```

Depois (modular):

```python
def ler_dados():
    nome = input("Nome: ")
    n1 = float(input("N1: "))
    n2 = float(input("N2: "))
    return nome, n1, n2

def calcular_media(n1, n2):
    return (n1 + n2) / 2

def verificar_status(media):
    return "Aprovado" if media >= 7
        else "Reprovado"

def processar():
    nome, n1, n2 = ler_dados()
    media = calcular_media(n1, n2)
    status = verificar_status(media)
    print(f"{nome}: {media} - {status}"
        )
```

Recapitulação

O que Aprendemos Hoje

Funções em Python:

1. **def**: Definir funções
2. **Parâmetros**: Entrada de dados
3. **Argumentos**: Valores passados
4. **return**: Devolver resultado
5. **Escopo local**: Variáveis dentro da função
6. **Escopo global**: Variáveis fora
7. **Docstrings**: Documentação
8. **Modularização**: Código organizado

Você agora sabe criar código reutilizável e organizado!

Por que usar funções?

Organização:

- Código limpo
- Fácil de entender
- Estruturado

Manutenção:

- Alterar em um lugar
- Menos bugs
- Fácil de corrigir

Reutilização:

- Escrever uma vez
- Usar várias vezes
- Compartilhar com outros

Testabilidade:

- Testar isoladamente
- Garantir qualidade
- Depurar facilmente

Próximos Passos

O que vem pela frente:

- **Módulos**: Organizar funções em arquivos
- **Bibliotecas**: Usar código de terceiros
- **Exceções**: Tratamento de erros
- **Arquivos**: Ler e escrever dados
- **Projetos maiores**: Integrar tudo!

Continue praticando! Funções são fundamentais para tudo.

Desafio para Casa

Projetos para praticar:

1. **Biblioteca Matemática:**
   - Crie funções: potencia, raiz, fatorial
   - MDC, MMC, primo
   - Fibonacci
2. **Validadores:**
   - Validar CPF
   - Validar email
   - Validar telefone

Dúvidas e Encerramento

Momento para perguntas!

Contato:

joao.quentino@dcomp.ufs.br

Obrigado!

Próxima aula: Módulos e Bibliotecas!